

Create executable files that run under monitoring by ASSIST09 or similar on MC09 on FLEX09.

 flexonsbd.blogspot.com/2023/08/flex09mc09assist09.html

Wednesday, August 16, 2023

Create an executable file that runs under the supervision of ASSIST09 or similar on MC09 on FLEX09.

Create an executable file that runs in an environment without FLEX09 using MC09 on FLEX09.

FLEX09 has a highly functional and easy-to-use assembler called ASMB, which I find very convenient. It also includes the well-known integer C compiler MC09.

While executable files created with ASMB can certainly be migrated and run in environments without FLEX09, and I often use them in that way, I had mistakenly believed that the executable files generated by MC09 were closely related to FLEX09 and could not be run in other environments.

However, there used to be a website called "Small Language" by wnoy, where the MPB6809 microcontroller board and a monitor program for it were made public.

(This site no longer exists, but someone who saved its contents has shared it. You can find it by searching for "MPB6809".)

This site also features other projects such as Forth ports and TinyBasic analyses, but they have also compiled and ported PL/0 using an MC09.

I also entered this monitor program, burned it to ROM, and ran PL/0 on my homemade 6809 microcontroller. PL/0 is not a compiler intended for practical use, so I didn't particularly intend to continue using it, but I was interested in the fact that an executable file compiled with MC09 was ported to a microcontroller called MPB6809.

It's obvious that a program that doesn't use file I/O that depends on FLEX09 will work even without FLEX09, but this was the first time I'd seen a real-world example. I discovered this site more than ten years ago, and since then, I've occasionally tried modifying the source listings compiled with MC09 to see if they would run on my own microcontroller.

The PL/0 source code needs to be significantly rewritten before it can be compiled on MC09, so I haven't been able to compile PL/0 yet. However, I've recently managed to get the other programs working somehow.

The microcontrollers used for the operational tests are shown in the image below, with the 63B09 and 68B02 operating at 2MHz being used interchangeably. For the 6809, a custom-made monitor and ASSIST09 are loaded into ROM, while for the 6802, a custom-made monitor and MIKBUG2 are loaded into ROM. Using a virtual drive creation software called FlexDrvWin.exe, FLEX09 for the 6809 and FLEX2 for the 6800 are running.



Board microcontroller used

Procedure for creating executable files on MC09

The result of compiling with MC09 is output as an assembly listing called C.OUT. This is then loaded into an assembly listing called C.TXT, which is included as standard with MC09, and the entire code is assembled to generate an executable file.

MC09 0000.C --> The assembly list C.OUT is created.

ASMB C.TXT 0000.CMD --> C.OUT is read into C.TXT, and the executable file 0000.CMD is generated.

You should be able to fix it by rewriting C.TXT.

This C.TXT file contains console I/O and file I/O, so I should be able to modify the console I/O to suit my environment and delete the file I/O since it's not needed. However, I encountered a lot of errors and it was a continuous process of trial and error, but I finally managed .

Of course, programs that require FLEX09 file I/O won't work, but programs that don't require file I/O can be created with MC09 and run on a microcontroller or similar device. As an example, I've created a memory dump program called mdump.c.

I created two versions: one for ASSIST09 and one for my custom monitor.

I created two files: C_SBC.TXT for my custom monitor installed on my homemade microcontroller, and C_ASSIST.TXT for Motorola's ASSIST09 monitor.

By using these files in place of the standard C.TXT, you can obtain executable files that run on your custom monitor or ASSIST09.

MC09 -O1.C.OUT oooo.C --> The assembly list C.OUT is created on drive 1.

ASMB C_SBC oooo.BIN +YLS or ASMB C_ASSIST oooo.BIN +YLS --> The executable file oooo.BIN will be created on drive 1.

Extract this executable file from the FLEX09 disk, install it on the microcontroller, and run it starting from \$100.

(Although the C.TXT file contains "ORG \$100," meaning the executable created with MC09 is supposed to run from \$100, it's actually not address-dependent code, so it can run from any address.)

The C.SBC.TXT and C.ASSIST.TXT files I created are shown below. Basically, only the I/O-related addresses are different, but the ASSIST09 I/O routine uses SWI instructions, while my custom monitor uses standard JSR instructions, so the call methods are adjusted accordingly.

The following is a list of C.ASSIST.TXT and C.SBC.TXT files.

C_SBC.TXT

C_ASSIST.TXT

The section marked (omitted below) is the same as line 192 onwards in the standard C.TXT file.

For reference, here is a sample mdump.c file I created. Programs like this, which don't involve file I/O, can run even without FLEX09.

mdump.c

The resulting mdump.bin file was about 4.2KB in size. While the size is larger, it takes less time to create than writing it in assembly language, so if size isn't a major issue, there are advantages to using MC09.

For reference, here are the specific steps I took to create and run the executable file.

1. Create the source program using an editor on a Windows PC.
2. Drag and drop the source code onto the image file (.DSK or .D77) in FlexDrvWin.exe.
3. Compile with MC09 and assemble with ASMB.
4. Load the resulting executable file (.BIN) from the image file onto the Windows PC.
5. Convert the binary executable file to Motorola format (.MOT) using the custom conversion software CvtMotHexBin.exe.
6. Load the Motorola-formatted executable file using the monitor's Load command.
7. Execute using the Go command on the monitor.

That's all for now, but I've uploaded the files I created and used (C_ASSIST.TXT, C_SBC.TXT, the latest version of FlexDrvWin.exe, CvtMotHexBin.exe, and the sample mdump.c) to [OneDrive](#) . Note that the creation procedures for FLEX09 and MC09 were introduced in a previous blog post.